FAMILY SOCIAL NETWORK
PHASE 1 — CORE FAMILY NETWORK

Complete Phase 1 implementation specification — divided into 5 stages

1. Phase 1 Purpose

Phase 1 is the complete core family-network foundation of the project. Its purpose is to allow users to create profiles, establish and verify family relationships, handle conflicting relationship claims, represent relatives who do not yet have accounts, build a relationship graph, explore that graph through an interactive family tree, calculate how two connected people are related, and highlight the relationship path between them.

Phase 1 must work as a complete standalone family relationship system. It does not depend on later features.

2. Phase 1 Scope

- Authentication
- User Profiles
- Family Relationship Requests
- Relationship Verification
- Relationship Conflict Handling
- Dummy Relative Profiles
- Claim Dummy Profile
- Family Relationship Graph
- Interactive Family Tree
- How Are We Related?
- Relationship Path Highlighting
- Relative Profile and Relationship Information
- Relationship Notifications
- Basic Family Search / Explore Family

3. Phase 1 Divided into 5 Stages

Stage	Name	Main Goal
Stage 1	Authentication & User Profiles	Create secure accounts and complete family-oriented profiles.
Stage 2	Relationship Requests & Verification	Allow users to establish and verify family relationships.
Stage 3	Dummy Profiles & Conflict Handling	Handle relatives without accounts and conflicting relationship claims.
Stage 4	Family Graph & Interactive Tree	Build and explore the family relationship graph visually.
Stage 5	How Are We Related? & Path Highlighting	Calculate, explain and visually highlight relationships.

4. Phase 1 Overall Flow

```
STAGE 1
Authentication + Profile
        ↓
STAGE 2
Relationship Request + Verification
        ↓
STAGE 3
Dummy Profiles + Conflict Handling
        ↓
STAGE 4
Family Graph + Interactive Family Tree
        ↓
STAGE 5
How Are We Related? + Path Highlighting
```

STAGE 1 — AUTHENTICATION & USER PROFILES

Stage 1 creates the identity and profile foundation required before users can build family relationships.

5. Stage 1 Objectives

- Allow a person to create an account.
- Allow a registered user to log in securely.
- Maintain the user's account state.
- Create a family-oriented user profile.
- Store profile information separately from relationship records.
- Prepare the user identity that will later become a node in the family graph.

6. Authentication

- User registration
- User login
- JWT-based authentication
- Protected routes
- Logout/session handling
- Password security using appropriate password hashing
- Authentication middleware on protected backend routes

7. User Profile Fields

Field	Required/Optional	Purpose
Name	Required	Display identity in the family network.
Region of Origin	Defined profile field	Family/profile information such as Andhra Pradesh, Bengal, Punjab, etc.
Mother Tongue	Defined profile field	Family/profile information.
Family Surname / Family Name	Defined profile field	Family name information.
Surname Origin	Optional	Additional surname history information.
Caste/Subcaste	Optional and private	Optional profile information; not relationship proof.

Caste/subcaste must remain optional and private by default. Region of origin, mother tongue, surname information, and caste/subcaste are profile information only. They must not be used as automatic proof that two people are relatives.

8. Stage 1 User Flow

1. User opens the registration page.
2. User creates an account.
3. System validates the registration data.
4. System creates the user account.
5. User logs in.
6. System creates/authenticates the user session using JWT.
7. User completes or edits their profile.
8. Profile becomes available for the family-network system.
9. The user can proceed to adding family relationships.

9. Stage 1 Data

```
User
├── id
├── name
├── email / login identifier
├── passwordHash
├── regionOfOrigin
├── motherTongue
├── familySurname
├── surnameOrigin (optional)
├── casteSubcaste (optional/private)
└── accountState
```

10. Stage 1 Completion Criteria

- A new user can register.
- A registered user can log in.
- Protected APIs reject unauthenticated access.
- A logged-in user can view and edit their profile.
- Profile information is stored correctly.
- The user has a stable internal ID that can be used in relationship records.

STAGE 2 — RELATIONSHIP REQUESTS & VERIFICATION

Stage 2 introduces the actual family relationship system. Users can connect to other accounts by explicitly selecting a supported family relationship.

11. Stage 2 Objectives

- Allow a user to search for another person.
- Allow the user to select a relationship type.
- Send a relationship request.
- Allow the recipient to accept or reject the request.
- Track relationship status.
- Allow accepted relationships to become verified within the platform.
- Create the relationship edge only according to the application's defined relationship rules and status.

12. Supported Relationship Types

Relationship Type	Related/Inverse Concept
Father	Child
Mother	Child
Parent	Child
Son	Father/Mother

Daughter	Father/Mother
Brother	Sibling
Sister	Sibling
Uncle	Nephew/Niece
Aunt	Nephew/Niece
Grandfather	Grandchild
Grandmother	Grandchild

The application should use explicit relationship records and defined inverse mappings rather than assuming relationships from unrelated profile attributes.

13. Relationship Request Flow

```
User A
  ↓
Search User B
  ↓
Select Relationship Type
  ↓
Send Relationship Request
  ↓
User B receives notification
  ↓
Accept / Reject
  ↓
Accepted relationship
  ↓
Verification status
```

14. Relationship States

State	Meaning
Pending	Relationship request has been sent but not finalized.
Accepted	Recipient accepted the relationship request.
Rejected	Recipient rejected the relationship request.
Verified	Relationship is confirmed within the platform according to its verification process.
Disputed	A conflicting relationship claim exists and needs to remain marked as disputed.

15. Relationship Verification

Verification is platform-level confirmation of the relationship record. It is not legal identity verification or legal proof of a family relationship.

The system should keep relationship status explicit so the family graph does not treat every unconfirmed claim as equally established.

16. Relationship Data Model

```
Relationship
├── id
├── user_id
├── related_user_id
├── relationship_type
├── status
├── created_at
└── updated_at
```

17. Relationship Request Data

```
RelationshipRequest
├── id
├── requester_id
├── recipient_id
├── relationship_type
├── status
├── created_at
└── updated_at
```

18. Stage 2 Completion Criteria

- A user can find another registered user.
- A user can select a supported family relationship.
- A relationship request can be sent.
- The recipient receives a relationship notification.
- The recipient can accept or reject the request.
- The relationship state is stored.
- An accepted/verified relationship can become an edge in the family graph.

STAGE 3 — DUMMY PROFILES & CONFLICT HANDLING

Stage 3 solves two important real-world family-network problems: relatives who do not yet have accounts and conflicting relationship claims.

19. Dummy Relative Profiles

A user can create a dummy profile when a relative does not have an account on the platform. The dummy profile exists to represent the relative in the family network and family tree.

20. Dummy Profile Information

- Internal platform ID
- Name
- Basic information required by the application
- Dummy/account state
- Relationship connections

21. Dummy Profile Restrictions

- A dummy profile cannot post.
- A dummy profile cannot function as an active social account.
- A dummy profile cannot independently join or create social groups.
- The dummy profile exists for family relationship and tree purposes until it is claimed.

22. Claim Dummy Profile

When the real person later creates or claims the profile, the existing dummy profile can be converted into a real account while retaining the established family relationships.

```
Dummy Relative
      ↓
Real Person Claims Profile
      ↓
Account Becomes Active
      ↓
Existing Family Relationships Remain Connected
```

23. Dummy Profile Data Model

```
DummyProfile / User Account State
├── id
├── name
├── accountType = DUMMY / REAL
├── basicProfileData
└── claimState
```

A separate permanent dummy-profile table is optional at implementation level if the User model can safely represent the account state. The important requirement is that the profile has a stable internal ID and can later be claimed.

24. Relationship Conflict Handling

The platform must not silently replace an existing family relationship when another conflicting claim is submitted.

```
Existing:
Phani → Father → Ramesh

New claim:
Phani → Father → Suresh

System result:
Keep the existing relationship record.
Create/track the new claim as disputed or pending according to the defined workflow.
```

25. Conflict States

- Pending
- Accepted
- Rejected
- Disputed

26. Conflict Handling Flow

10. User submits a relationship claim.
11. System checks existing relationship records relevant to the relationship type.
12. If no conflict exists, the request follows the normal relationship flow.
13. If a conflicting relationship exists, the system does not silently overwrite it.
14. The new claim is marked as disputed/pending according to the application workflow.
15. The relevant users can see the relationship status.
16. The final graph should use the application's defined accepted/verified relationship rules.

27. Stage 3 Completion Criteria

- A user can create a dummy relative.
- The dummy relative appears in the family relationship network.
- Dummy profiles cannot behave as active social accounts.
- A real person can claim the dummy profile.
- Existing relationships remain connected after a successful claim.
- Conflicting relationship claims are detected.
- Conflicting claims are not silently used to overwrite existing data.
- Relationship records clearly show their status.

STAGE 4 — FAMILY GRAPH & INTERACTIVE FAMILY TREE

Stage 4 turns the relationship records into the central family graph and provides the interactive family-tree experience.

28. Stage 4 Objectives

- Represent users and relatives as graph nodes.
- Represent family relationships as graph edges.
- Generate the family tree from relationship data.
- Allow users to explore family branches.
- Open a person's profile from the tree.
- Show relationship labels.
- Show relationship status information.
- Represent dummy profiles.
- Support the relationship path functionality required in Stage 5.

29. Family Graph Model

```
Users = Nodes
Relationships = Edges

Example:
Phani → Father → Ramesh
Phani → Mother → Sunita
Ramesh → Father → Mohan
Sunita → Brother → Raj
Raj → Son → Rahul
```

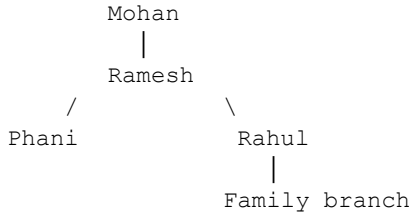
30. Why the Tree Is Derived

The application should not maintain a separate rigid family-tree structure with fields such as fatherId, motherId, uncleId, brotherId and sisterId. The relationship graph is the source of truth. The interactive family tree is generated from the graph.

31. Family Tree Capabilities

- Zoom
- Pan
- Expand family branches
- Collapse family branches
- Click a person
- Open the person's profile
- Display relationship labels
- Display verified/unverified status where applicable
- Display dummy-profile state
- Explore different family branches
- Select a person for relationship-path viewing

32. Example Family Tree



33. Explore Family

Explore Family is the basic family-navigation feature. A user can move from their own profile to connected relatives and continue through available family branches using the graph/tree.

- Start from the user's profile.
- View direct relatives.
- Open a relative's profile.
- Move to another connected branch.
- Return to the user's family-tree position.
- Use the tree to explore connected people.

34. Relative Profile and Relationship Information

A person's profile can display relationship information when the platform has a known relationship path.

```
Rahul Sharma
Possible Cousin / Cousin

[View Relationship in Family Tree]
[How Are We Related?]
```

35. Stage 4 Completion Criteria

- Relationship records can be converted into a graph representation.
- The family tree is generated from the relationship graph.
- Users can zoom and pan through the tree.
- Users can expand/collapse branches.
- Users can open profiles from tree nodes.
- Relationship labels can be displayed.
- Dummy profiles can appear in the tree.
- Relationship status can be represented.
- The tree provides the interface required for relationship-path highlighting.

STAGE 5 — HOW ARE WE RELATED? & RELATIONSHIP PATH HIGHLIGHTING

Stage 5 completes the Phase 1 family-network experience. It uses graph traversal to calculate the connection between two people and then visually highlights that connection in the family tree.

36. Stage 5 Objectives

- Find a path between two connected people.
- Translate the path into understandable relationship steps.
- Display the resulting relationship information.
- Open the relationship path from a person's profile.
- Highlight the exact path in the interactive family tree.

37. How Are We Related?

The system uses the family relationship graph to find a path between the current user and another connected person.

```
Example:
Phani
  ↓
```

```
Mother
  ↓
Mother's Brother
  ↓
Brother's Son
  ↓
Rahul

Relationship result:
Rahul is Phani's cousin.
```

38. Graph Traversal

A graph traversal algorithm such as Breadth-First Search (BFS) can be used to find a path between two nodes. The stored relationship edges provide the actual evidence for the path.

```
Start Node = Current User
Target Node = Selected Relative

BFS / graph traversal
        ↓
Find connected path
        ↓
Convert edges into relationship steps
        ↓
Display relationship explanation
```

39. Relationship Path Highlighting

When a user selects 'View Relationship in Family Tree', the system opens the tree and highlights the nodes and edges that form the relationship path.

```
Example:

YOU
 |
 ▼
FATHER
 |
 ▼
FATHER'S BROTHER
 |
 ▼
COUSIN

Only the relevant path is highlighted in the tree.
```

40. Stage 5 User Flow

17. User opens another connected person's profile.
18. The system identifies the known relationship information.
19. User selects 'How Are We Related?' or 'View Relationship in Family Tree'.
20. Backend finds a valid graph path between the two people.
21. Backend returns the relationship nodes/edges in order.
22. Frontend opens the family tree at the relevant area.
23. The path nodes and edges are highlighted.
24. The relationship explanation is displayed alongside the tree.

41. Relationship Path Response Structure

```
Relationship Path
├── sourceUser
├── targetUser
├── nodes[]
├── relationships[]
├── relationshipLabel
└── pathStatus
```

42. Stage 5 Completion Criteria

- The system can calculate a path between connected people.
- The path contains the correct relationship edges from the graph.
- The relationship can be displayed in understandable terms.
- The user can open the relationship path from a profile.
- The family tree opens with the relevant path highlighted.
- The highlighted path corresponds to the relationship explanation.

43. Notifications Across Phase 1

Notifications are part of Phase 1 because users need to know when relationship actions require their attention.

Notification	Trigger
Relationship Request	Another user sends a relationship request.
Request Accepted	A requested relationship is accepted.
Relationship Status Update	The relationship status changes.
Relationship Dispute	A conflicting relationship claim is recorded.
Dummy Profile Claim	A dummy profile is claimed/converted into an active account.

44. Phase 1 Backend Modules

Module	Responsibilities
Auth	Registration, login, JWT, protected access
Users	User profile creation, viewing and editing
Relationships	Relationship requests, status and relationship records
Verification	Platform relationship verification state
Conflicts	Detect and track conflicting relationship claims
Dummy Profiles	Create and claim relatives without accounts
Notifications	Relationship-related notifications
Family Graph	Build graph representation from relationship records
Family Tree	Return data needed by the interactive tree
Relationship Path	Find path and relationship explanation

45. Phase 1 API Areas

These are the API areas required by the defined Phase 1 functionality. Exact route names can be finalized during implementation.

```
AUTH
POST   /auth/register
POST   /auth/login
POST   /auth/logout
```

```
USERS
GET    /users/me
PATCH  /users/me
GET    /users/search
GET    /users/:id

RELATIONSHIPS
POST   /relationships/requests
GET    /relationships/requests
PATCH  /relationships/requests/:id
GET    /relationships
GET    /relationships/:id

DUMMY PROFILES
POST   /dummy-profiles
GET    /dummy-profiles/:id
POST   /dummy-profiles/:id/claim

GRAPH / TREE
GET    /family/graph
GET    /family/tree

RELATIONSHIP PATH
GET    /relationships/path/:userId

NOTIFICATIONS
GET    /notifications
PATCH  /notifications/:id/read
```

46. Phase 1 Database Relationships

```
User
  │
  ├── RelationshipRequest
  │
  ├── Relationship ────────── User
  │
  ├── Notification
  │
  └── Dummy/Account State

Relationship records
        ↓
Family Graph
        ↓
Interactive Family Tree
        ↓
Relationship Path
```

47. Phase 1 Frontend Areas

- Registration page
- Login page
- User profile page
- Edit profile page
- User search

- Add relative / relationship request interface
- Incoming relationship requests
- Relationship status display
- Notifications
- Create dummy relative
- Claim dummy profile
- Family tree page
- Family member profile
- Explore Family
- How Are We Related? interface
- Relationship path highlighted tree view

48. Phase 1 Technology Structure

```
Frontend
React + TypeScript + Tailwind CSS
        ↓ REST API
Backend
Node.js + Express
        ↓
Prisma ORM
        ↓
PostgreSQL

Authentication
JWT

Validation
Zod
```

49. Phase 1 Core Database Schema

Model	Core fields / responsibility
User	id, name, login information, profile fields, account state
Relationship	id, user_id, related_user_id, relationship_type, status, timestamps
RelationshipRequest	id, requester_id, recipient_id, relationship_type, status, timestamps
Notification	id, recipient_id, type, related entity/reference, read state, timestamps
Dummy Profile	Can be represented by user/account state or dedicated structure; stores internal ID, name/basic data and claim state

50. Phase 1 Security and Data Rules

- Passwords must not be stored as plain text.
- Protected endpoints must require authentication.
- A user must not be able to edit another user's profile without an authorized workflow.
- Relationship changes must validate that the requester is allowed to perform the action.
- Relationship conflicts must not silently overwrite existing records.
- Sensitive profile information such as optional caste/subcaste must have controlled visibility.

- Government identity numbers such as Aadhaar are not part of the Phase 1 data model.
- Family relationship information should be exposed only according to the defined access/visibility rules.

51. Phase 1 Implementation Order

25. Set up the React + TypeScript + Tailwind frontend.
26. Set up the Node.js + Express backend.
27. Set up PostgreSQL and Prisma.
28. Implement User model and authentication.
29. Implement user profile APIs and UI.
30. Implement relationship request model and APIs.
31. Implement relationship acceptance/rejection.
32. Implement relationship verification state.
33. Implement notifications for relationship actions.
34. Implement conflict detection and disputed status.
35. Implement dummy profiles.
36. Implement dummy-profile claim flow.
37. Implement family graph generation.
38. Implement interactive family-tree UI.
39. Implement user/profile navigation from the tree.
40. Implement graph path calculation.
41. Implement How Are We Related?
42. Implement relationship-path highlighting.
43. Test the complete Phase 1 flow.

52. Phase 1 Testing Scenarios

Test	Expected result
Register user	Account is created and receives internal ID.
Login	Authenticated session/JWT is issued.
Edit profile	User's allowed profile fields update.
Send relationship request	Recipient receives request.
Accept request	Relationship becomes accepted according to rules.
Reject request	Relationship is not established.
Verify relationship	Relationship status becomes verified.
Conflicting parent claim	Existing relationship is not silently replaced; conflict is tracked.
Create dummy relative	Dummy relative appears in the family graph/tree.
Claim dummy profile	Dummy profile becomes associated with the real account while relationships remain.
Open family tree	Connected family members are visualized.
Open relative profile	Relationship information is shown when known.
How Are We Related?	A valid graph path is returned for connected users.
Highlight relationship	Correct nodes/edges are highlighted in the family tree.
Unauthorized access	Protected operation is rejected.

53. Phase 1 Final Deliverable

At the end of Phase 1, the project must provide a working family relationship network. A user can create an account and profile, connect relatives through relationship requests, establish relationship status, handle conflicting claims, represent relatives who do not have accounts, claim dummy profiles, explore the resulting

family graph through an interactive family tree, open family-member profiles, calculate how connected people are related, and highlight the exact relationship path in the tree.

54. Phase 1 Completion Checklist

- ☐ Authentication works
- ☐ User profile works
- ☐ User search works
- ☐ Relationship requests work
- ☐ Accept/reject works
- ☐ Relationship verification state works
- ☐ Relationship conflicts are detected and tracked
- ☐ Dummy relative creation works
- ☐ Dummy profile claim works
- ☐ Family relationship graph works
- ☐ Interactive family tree works
- ☐ Family tree supports zoom/pan
- ☐ Family branches can be explored
- ☐ Person nodes open profiles
- ☐ Relationship labels are shown
- ☐ Relationship notifications work
- ☐ How Are We Related? works
- ☐ Relationship path is calculated
- ☐ Relationship path is highlighted in the family tree
- ☐ Phase 1 complete flow works from account creation to relationship exploration

55. Phase 1 in One View

